

理解確認と議論

実行の理由を説明する

情報システム学科 3年・選択・2単位

今日の到達目標

- 実行前に結果と理由を予測できる
- 環境と評価順序を図で説明できる
- 他者の説明を反例で確かめられる

前回との接続

Lispで書いた評価器を段階的に開発した。

今日は新機能よりも、動く理由の説明を確かめる。

自分の実装と参照実装の違いも議論の材料になる。

引用と `#f` の分岐は最小評価器、比較を含む分岐と関数は対応ホストか完成参照で確かめる。使う版を記録し、自作版の対応範囲と区別する。

議論の進め方

個人で予測を書いてから、ペアで理由を比較する。

コードを実行し、予測との違いを調べる。

説明を修正し、その説明を崩せる入力を探す。

授業内は2～3のテーマを選び、この流れを最後まで行う。残る問いは予復習で検討する。

証拠の持ち寄り

各自、成功例1個と説明しづらい例1個を持ち寄る。

入力、期待、実際、関連コードを分けて示す。

他者が再実行できるコマンドを添える。

予測1 引用

```
(quote (+ 2 3))  
(+ 2 3)
```

各結果を実行せずに書く。

どの名前を検索するかも説明する。

議論1 評価の停止

相手の説明に、引用された未定義名を加える。

`(quote missing)` は成功するだろうか。

成功する理由を、読取と評価の区別で説明する。

予測2 分岐

```
(if #f (/ 1 0) 8)  
(if (= 1 1) 9 missing)
```

結果だけでなく、評価しない部分に印を付ける。

議論2 ifの実装

通常関数と同じ方法で `if` を実装できるか。

相手の案を、非選択枝のゼロ除算で確かめる。

「特殊形式」の説明を自分の言葉に直す。

予測3 クロージャ

```
(define make-adder  
  (lambda (x) (lambda (y) (+ x y))))  
(define plus2 (make-adder 2))  
(plus2 7)
```

x がどこに残るかを図にする。

議論3 環境の寿命

`make-adder` の呼出しが終わった後、なぜ `x` を使えるか。

保存する環境が呼出し元の環境だった場合の反例を提案する。

ペアで異なる値の加算器を作って検証する。

予測4 名前の隠蔽

```
(define x 100)
(define f (lambda (x) (+ x 1)))
(f 4)
x
```

各名前検索が到達する環境を記入する。

議論4 共有と分離

すべての関数呼出しで同じ辞書を使う案を検討する。

どの式で外側の `x` を壊してしまうか。

実行前に予測し、検証後に説明を更新する。

追跡の観察

```
cd examples/13-language-review  
python3 lisp.py --trace --meta --eval "(+ 1 (* 2 3))"
```

結果は 7。標準エラーの [host depth=N] と [meta depth=N] で評価の層を区別する。

出力された順序と、自分の木を照合する。

議論5 追跡の限界

追跡ログに名前が出て、その環境の中身が分かるとは限らない。

不足する証拠を1つ挙げ、追加する方法を考える。

ログの量と読みやすさも比較する。

予測5 二重の評価

ホストが引用された対象式を読み、その後Lisp製評価器が処理する。

どの層で `quote` が1つ外れるか、順に書く。

対象の記号をホストが先に検索する誤りを説明する。

議論6 セルフホスティング

Lisp製評価器のどの処理が、まだPythonホストへ依存するか。

対象の `eval`、読取、入出力を別々に確かめる。

達成したことを、実行コマンドで裏付ける。

相互質問の準備

自分が説明できる箇所から、問いを1個作る。

正答を当てる問いに加え、理由を追う追質問を用意する。

相手のコードを無断で変更せず、小さな入力で確かめる。

設計レビュー

相手の評価器から、変更が複数箇所へ広がる機能を探す。

環境操作を1箇所にまとめる案を説明する。

変更前後で同じ受入例が通るか検討する。

不具合の切り分け

「構文が読めない」「名前がない」「適用できない」を分類する。

自分の不具合を分類し、最小の再現入力へ縮める。

ペアは別の分類にならないか問い直す。

AI回答の検討

AIへ同じ実装の説明を求める。

説明中の環境・評価順序を、実行ログと比較する。

誤った箇所があれば、反例を示して修正する。

理解の記録

最初の予測、観察した事実、修正後の説明を残す。

未解決の問いに、次の検証入力を添える。

動作成功だけで理解したと判断しない。

到達の確認

初めて見る小さな式を説明できるか。

異なる実装の誤りを検出する反例を作れるか。

相手の指摘を受け、自分の説明を更新できるか。

まとめと次回

議論で曖昧な環境や評価順序を特定した。

次回はその理解を使ってLisp製評価器とREPLを仕上げる。

未解決の点を開発順序に反映する。

参考

- 本教材: `examples/13-language-review/`
- 第10～12回の演習ログと仕様
- SICP 評価器

今日の主資料は、自分と相手の実行結果。